# adapter.py — technical

`src/moe_pruning/models/switch_transformers/adapter.py · 371 lines · moe_pruning.models.switch_transformers.adapter`

## Purpose

Adapter for HF `SwitchTransformersForConditionalGeneration` (transformers 5). Experts are separate `SwitchTransformersDenseActDense` modules in a `ModuleDict` (`expert_0` ... `expert_{E-1}`).

## Data and structure

- Layer = `SwitchTransformersLayerFF`: `out = h + dropout(SparseMLP(LayerNorm(h)))`.
- SparseMLP: `router(x)` -> (one-hot dispatch `[B,S,E]` with capacity applied, top-1 prob `[B,S,1]`, logits); `experts(x, dispatch, probs)` -> sum of `expert(x) * prob` per token.

## Formulas

```
p = softmax(W x) (float32); e* = argmax p; dispatch[t, e*] = 1 if cumsum over sequence of tokens to e* <= capacity

y_t = p[t, e*] * E_e*(x_t) (0 for capacity-dropped tokens)
```

## Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

### `class SwitchTransformersAdapter`

lines 55-371 · bases MoEAdapter

MoEAdapter for Hugging Face Switch Transformers.

#### `SwitchTransformersAdapter.discover_layers`

```
def discover_layers(self, model: nn.Module) -> list[MoELayerInfo]
```

lines 59-105 · raises RuntimeError

Find all sparse MoE layers by inspecting the loaded model.

1. Walk `named_modules()`; a sparse MLP is any module with `router` (having `classifier`) and a `ModuleDict` `experts`.
2. From the path `[prefix.]{encoder|decoder}.block.{i}.layer.{j}.mlp` derive the stack and layer id (`encoder.block.1`).
3. Validate expert keys are exactly `expert_0..expert_{n-1}` and that router size matches.
4. Read dimensions from any not-yet-replaced expert's `wi`/`wo` weights (works after pruning); capacity from the router.

#### `SwitchTransformersAdapter.get_moe_module`

```
def get_moe_module(self, model: nn.Module, layer: MoELayerInfo) -> nn.Module
```

lines 110-111

The sparse MoE module of a layer.

#### `SwitchTransformersAdapter.get_router`

```
def get_router(self, model: nn.Module, layer: MoELayerInfo) -> nn.Module
```

lines 113-114

The router module of a layer.

#### `SwitchTransformersAdapter.get_expert`

```
def get_expert(self, model: nn.Module, layer: MoELayerInfo, expert_id: int) -> nn.Module
```

lines 119-121

Module computing one expert (original, view or replacement).

### SwitchTransformersAdapter.set_expert

```
def set_expert(self, model: nn.Module, layer: MoELayerInfo, expert_id: int, module: nn.Module) ->
nn.Module
```

lines 123-131

Install `module` in the expert slot and return the previous module.

### SwitchTransformersAdapter.build_original_expert

```
def build_original_expert(self, model: nn.Module, layer: MoELayerInfo) -> nn.Module
```

lines 133-138

New (uninitialised) expert module of the model's original expert class.

### SwitchTransformersAdapter.expert_state_prefix

```
def expert_state_prefix(self, layer: MoELayerInfo, expert_id: int) -> str
```

lines 140-141

State-dict key prefix of an expert slot (ends with a dot).

### SwitchTransformersAdapter._dispatch_to_index

```
def _dispatch_to_index(dispatch: torch.Tensor, probs: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]
```

lines 156-161

Convert the one-hot capacity-masked dispatch mask into index/gate tensors.

1. `routed = dispatch.sum(-1) > 0`; index = argmax of dispatch where routed else -1; gate = prob * routed.
2. Returned as `[B, S, 1]` tensors (K = 1).

### SwitchTransformersAdapter.route

```
def route(self, model: nn.Module, layer: MoELayerInfo, moe_input: torch.Tensor, apply_mask: bool=True)
-> tuple[torch.Tensor, torch.Tensor]
```

lines 163-173

Return (expert_index [B,S,K], gate [B,S,K]) exactly as used by the layer.

1. Temporarily set `router._moe_pruning_mask_disabled = not apply_mask`, call the router, restore the flag, convert dispatch to index/gate.

### SwitchTransformersAdapter._masked_routing_hook

```
def _masked_routing_hook(router, _args, output)
```

lines 179-199

Re-do the HF top-1 + capacity assignment with masked experts excluded.

1. Router forward hook (registered with `prepend=True` so it runs before instrumentation hooks).
2. Does nothing if no experts are masked or the mask is disabled.
3. Recompute `probs = softmax(logits, float32)`, set masked columns to -1 for selection, argmax, one-hot, apply the same per-sequence capacity cumsum, gate = original prob of the chosen expert.
4. Returns the replacement `(dispatch, gate, logits)`; with no masked experts this equals the HF result exactly.

### SwitchTransformersAdapter.set_routing_mask

```
def set_routing_mask(self, model: nn.Module, layer: MoELayerInfo, expert_ids: list[int]) -> None
```

lines 201-213 · raises ValueError

Make `expert_ids` unreachable: their tokens go to the best remaining expert.

1. Validate ids; refuse to mask all experts; store the tuple on the router; register the hook once; remove it when the mask becomes empty.

### `SwitchTransformersAdapter.get_routing_mask`

```
def get_routing_mask(self, model: nn.Module, layer: MoELayerInfo) -> list[int]
```

lines 215-216

Experts currently made unreachable by the reroute mask.

### `SwitchTransformersAdapter.run_moe_layer`

```
def run_moe_layer(self, model: nn.Module, layer: MoELayerInfo, moe_input: torch.Tensor) -> torch.Tensor
```

lines 218-219

Execute only the sparse MoE module on `moe_input` ([B,S,D]).

### `SwitchTransformersAdapter.instrument`

```
def instrument(self, model: nn.Module, layers: list[MoELayerInfo], callback: RecordCallback) ->
Iterator[None]
```

lines 225-235

Call `callback` with a `MoEForwardRecord` per layer forward.

### `SwitchTransformersAdapter._register_layer_hooks`

```
def _register_layer_hooks(self, model: nn.Module, layer: MoELayerInfo, callback: RecordCallback) -> list
```

lines 237-298 · raises RuntimeError

1. `LayerFF` pre-hook: reset state, store residual `h`.
2. SparseMLP pre-hook: store MoE input `x`.
3. Router hook: store dispatch and probs (after any mask hook).
4. Each expert's forward hook: store its un-gated output rows (rows follow ascending token order of that expert).
5. SparseMLP forward hook: scatter expert rows back to token positions using the dispatch mask (row-count check), build the `MoEForwardRecord` and call the callback.

### `SwitchTransformersAdapter.describe`

```
def describe(self, model: nn.Module) -> dict
```

lines 303-371

Machine-readable architecture report.

1. Per-layer module classes, router size/dtype, expert parameter counts, plus model-level config (d_model, d_ff, sparse steps, capacity, jitter) and prose describing routing, capacity, gate application and padding.

## Errors raised

| Function | Exception | Message (f-string fields in braces) |
|---|---|---|
| `discover_layers` | RuntimeError | Unexpected sparse MoE location {path} |
| `discover_layers` | RuntimeError | Cannot determine encoder/decoder for {path} |
| `discover_layers` | RuntimeError | Unexpected expert keys in {path}: {keys} |
| `discover_layers` | RuntimeError | {path}: no original expert left to read dimensions from |
| `discover_layers` | RuntimeError | Router/expert count mismatch in {path} |
| `_check_expert_id` | IndexError | Expert {expert_id} out of range for {layer.layer_id} |
| `set_routing_mask` | ValueError | Cannot mask every expert of a layer |
| `_register_layer_hooks` | RuntimeError | {layer.layer_id}: expert {expert_id} returned {rows.shape[0]} rows for {tokens.numel()} dispatched tokens |
| `moe_hook` | RuntimeError | {layer.layer_id}: expert {expert_id} returned {rows.shape[0]} rows for {tokens.numel()} dispatched tokens |